GENETIC ALGORITHMS

What are GAs ?

• Genetic Algorithms (GAs) are adaptive heuristic search algorithm based on the evolutionary ideas of natural selection and genetics.

• Genetic algorithms (GAs) are a part of Evolutionary computing, a rapidly growing area of artificial intelligence. GAs are inspired by Darwin's theory about evolution - "survival of the fittest".

• GAs represents an intelligent exploitation of a random search used to solve optimization problems.

• GAs, although randomized, exploit historical information to direct the search into the region of better performance within the search space.

• In nature, competition among individuals for scanty resources results in the fittest individuals dominating over the weaker ones.

Introduction

Solving problems mean looking for solutions, which is best among others.
Finding the solution to a problem is often thought:

- In computer science and AI, as a process of search through the space of possible solutions. The set of possible solutions defines the search space (also called state space) for a given problem. Solutions or partial solutions are viewed as points in the search space.

- In engineering and mathematics, as a process of optimization. The problems are first formulated as mathematical models expressed in terms of functions and then to find a solution, discover the parameters that optimize the model or the function components that provide optimal system performance.

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

- **Why Genetic Algorithms?**

It is better than conventional AI ; It is more robust.

Unlike older AI systems, the GA's do not break easily even if the inputs changed slightly, or in the presence of reasonable noise.

While performing search in large state-space, multi-modal state-space, or n-dimensional surface, a genetic algorithms offer significant benefits over many other typical search optimization techniques like – linear programming, heuristic, depth-first, breath-first.

"Genetic Algorithms are good at taking large, potentially huge search spaces and navigating them, looking for optimal combinations of things, the solutions one might not otherwise find in a lifetime." Salvatore Mangano Computer Design, May 1995.

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

Genetic Algorithm: Fundamental

Genetic Algorithm (GA) is a search-based optimization technique based on the principles of Genetics and Natural Selection. It is frequently used to find optimal or near-optimal solutions to difficult problems which otherwise would take a lifetime to solve. It is frequently used to solve optimization problems, in research, and in machine learning.

What are Genetic Algorithms?

Nature has always been a great source of inspiration to all mankind. Genetic Algorithms (GAs) are search based algorithms based on the concepts of natural selection and genetics. GAs are a subset of a much larger branch of computation known as Evolutionary Computation.

In GAs, we have a pool or a population of possible solutions to the given problem. These solutions then undergo recombination and mutation (like in natural genetics), producing new children, and the process is repeated over various generations. Each individual (or candidate solution) is assigned a fitness value (based on its objective function value) and the fitter individuals are given a higher chance to mate and yield more "fitter" individuals. This is in line with the Darwinian Theory of "Survival of the Fittest".

Genetic Algorithms are sufficiently randomized in nature, but they perform much better than random local search (in which we just try various random solutions, keeping track of the best so far), as they exploit historical information as well.

Basic Concepts

Individual: An individual is a single solution. An individual groups together two forms of solutions as given below:

1. The chromosome which is the raw "genetic" information (genotype) that the GA deals.
2. The phenotype which is the expressive of the chromosome in the terms of the model.

A chromosome is subdivided into genes.

Genes: Genes are the basic "instructions" for building a GA. A chromosome is a sequence of genes. Genes may describe a possible solution to a problem, without actually being the solution. A gene is a bit string of arbitrary lengths. The bit string is a binary representation of number of intervals from a lower bound. A gene is the GA's representation of a single factor value for a control factor, where control factor must have an upper bound and a lower bound.

Fitness: The fitness of an individual in a GA is the value of an objective function for its phenotype. For calculating fitness, the chromosome has to be first decoded and the objective function has to be evaluated. The fitness not only indicates how good the solution is, but also corresponds to how close the chromosome is to the optimal one.

In case of multi criterion optimization, the fitness function is definitely more difficult to determine. In multi criterion optimization problems, there is often a dilemma as how to determine if one solution is better than another.

Population: A population is a collection of individuals. A population consist of a number of individuals being tested, the phenotype parameters defining the individuals and some information about the search space. The two important aspects of population used in GA are:

1. The initial population generation
2. The population size.

For each and every problem, the population size will depend on the complexity of the problem. It is often a random initialization of population.

Working Principle

Working principle of a GA is as follows –

We start with an initial population (which may be generated at random or seeded by other heuristics), select parents from this population for mating. Apply crossover and mutation operators on the parents to generate new off-springs. And finally these off-springs replace the existing individuals in the population and the process repeats, see figure 1.1. In this way genetic algorithms actually try to mimic the human evolution to some extent.

A generalized pseudo-code for a GA is explained in the following program –

Genetic Algorithm()

```
  initialize population
  find fitness of population
    while (termination criteria is reached) do
    parent selection
    crossover with probability pc
    mutation with probability pm
    decode and fitness calculation
    survivor selection
    find best
  return best
```

RGPVNOTES.IN

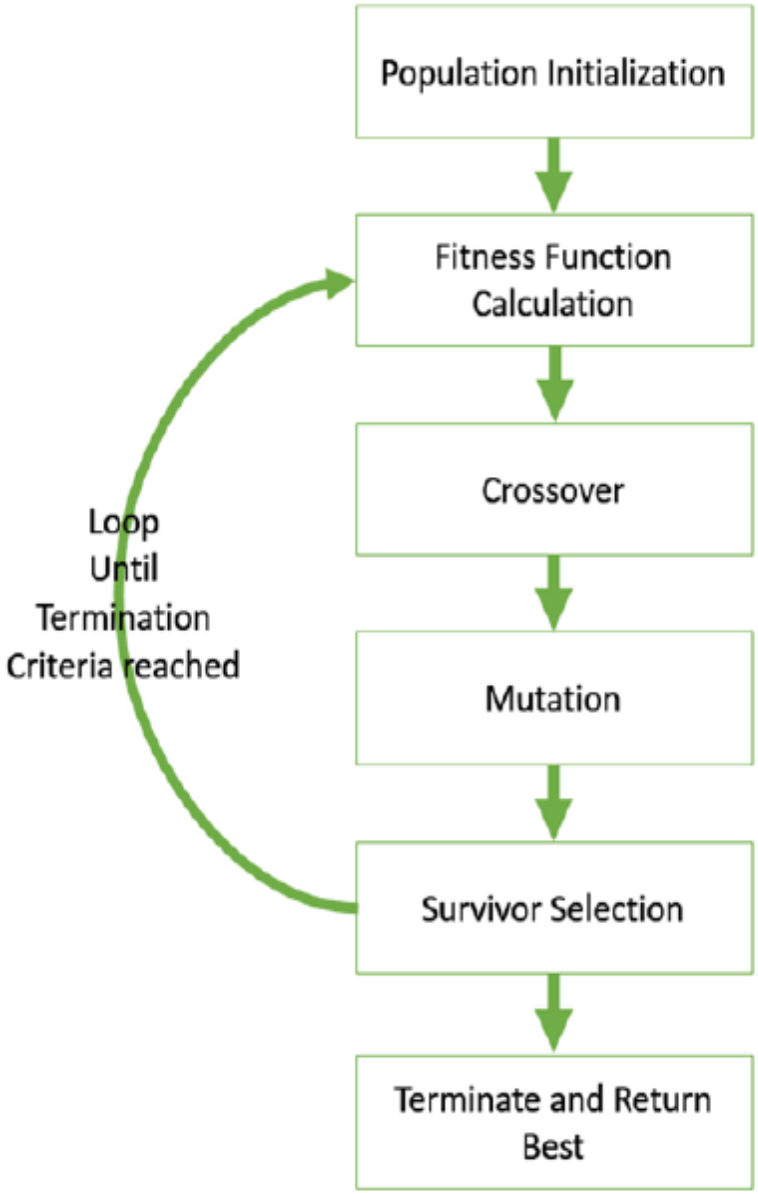


Fig 1.1: Flow Chart of GA

DR. VIS
, ACEIT

Encoding: Encoding is a process of representing individual genes. The process can be performed using bits, numbers, trees, arrays, lists or any other objects. The encoding depends mainly on solving the problems.

Types of encoding

One of the most important decisions to make while implementing a genetic algorithm is deciding the representation that we will use to represent our solutions. It has been observed that improper representation can lead to poor performance of the GA.

Therefore, choosing a proper representation, having a proper definition of the mappings between the phenotype and genotype spaces is essential for the success of a GA.

In this section, we present some of the most commonly used representations for genetic algorithms. However, representation is highly problem specific and the reader might find that another representation or a mix of the representations mentioned here might suit his/her problem better.

1. Binary Encoding

This is one of the simplest and most widely used representations in GAs. In this type of representation the genotype consists of bit strings.

For some problems when the solution space consists of Boolean decision variables – yes or no, the binary representation is natural. Take for example the 0/1 Knapsack Problem. If there are n

items, we can represent a solution by a binary string of n elements, where the xth element tells whether the item x is picked (1) or not (0).

2. Real Value Encoding

For problems where we want to define the genes using continuous rather than discrete variables, the real valued representation is the most natural. The precision of these real valued or floating point numbers is however limited to the computer.

3. Integer Encoding

For discrete valued genes, we cannot always limit the solution space to binary 'yes' or 'no'. For example, if we want to encode the four distances – North, South, East and West, we can encode them as **{0,1,2,3}**. In such cases, integer representation is desirable.

4. Permutation Encoding

In many problems, the solution is represented by an order of elements. In such cases permutation representation is the most suited.

A classic example of this representation is the travelling salesman problem (TSP). In this the salesman has to take a tour of all the cities, visiting each city exactly once and come back to the starting city. The total distance of the tour has to be minimized. The solution to this TSP is naturally an ordering or permutation of all the cities and therefore using a permutation representation makes sense for this problem.

Fitness Function

The fitness function simply defined is a function which takes a candidate solution to the problem as input and produces as output how "fit" our how "good" the solution is with respect to the problem in consideration.

RGPVNOTES.IN

Calculation of fitness value is done repeatedly in a GA and therefore it should be sufficiently fast. A slow computation of the fitness value can adversely affect a GA and make it exceptionally slow.

In most cases the fitness function and the objective function are the same as the objective is to either maximize or minimize the given objective function. However, for more complex problems with multiple objectives and constraints, an Algorithm Designer might choose to have a different fitness function.

A fitness function should possess the following characteristics –

- The fitness function should be sufficiently fast to compute.
- It must quantitatively measure how fit a given solution is or how fit individuals can be produced from the given solution.

In some cases, calculating the fitness function directly might not be possible due to the inherent complexities of the problem at hand. In such cases, we do fitness approximation to suit our needs.

The following image shows the fitness calculation for a solution of the 0/1 Knapsack. It is a simple fitness function which just sums the profit values of the items being picked (which have a 1), scanning the elements from left to right till the knapsack is full.

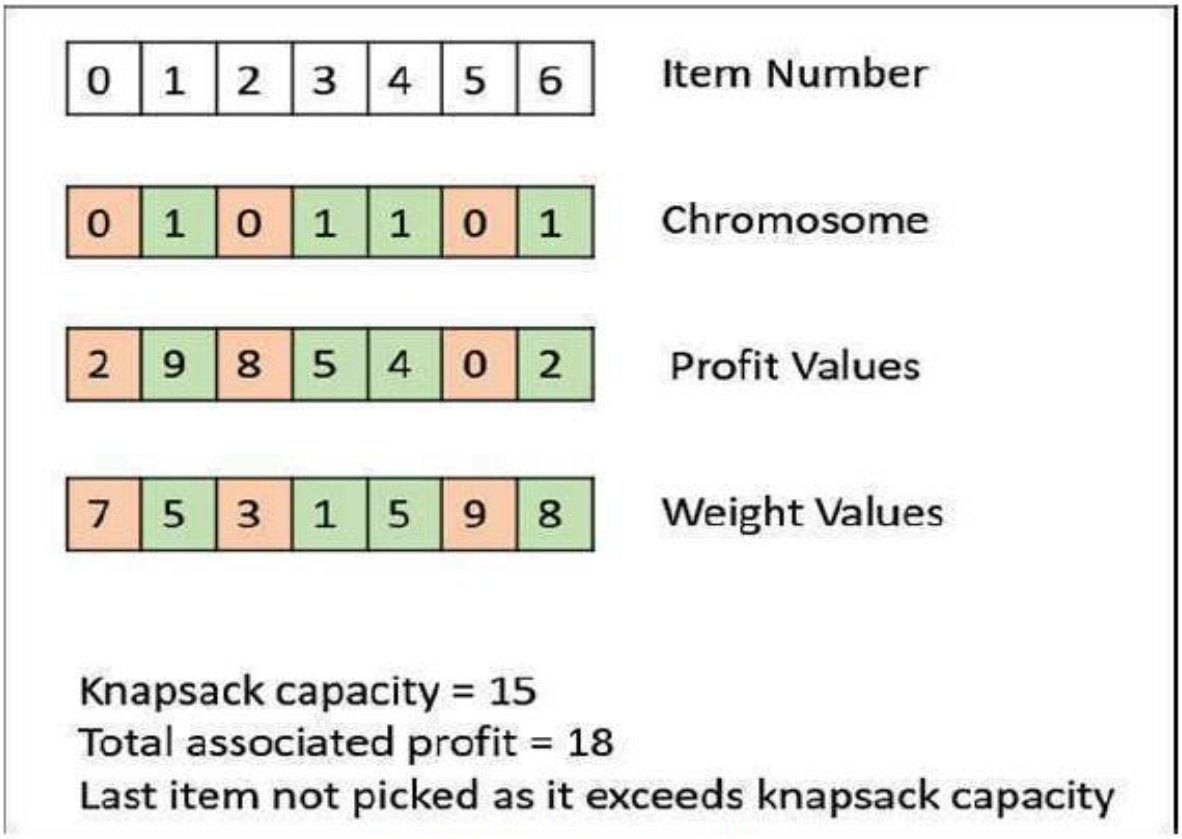


Fig 1.2: Fitness Function

DR. VISHAL SHRIVASTA

Reproduction / Selection

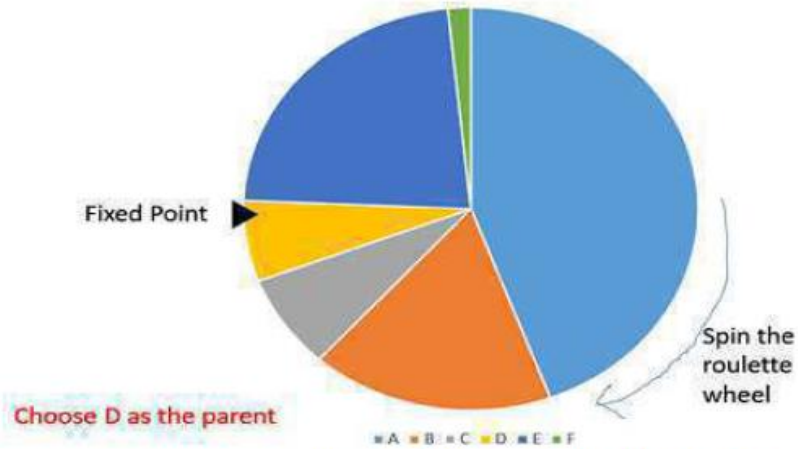
Fitness Proportionate Selection is one of the most popular ways of parent selection. In this every individual can become a parent with a probability which is proportional to its fitness. Therefore, fitter individuals have a higher chance of mating and propagating their features to the next generation. Therefore, such a selection strategy applies a selection pressure to the more fit individuals in the population, evolving better individuals over time.

Following are some of the selection strategies:

Roulette Wheel Selection

In a roulette wheel selection, the circular wheel is divided as described before. A fixed point is chosen on the wheel circumference as shown and the wheel is rotated. The region of the wheel which comes in front of the fixed point is chosen as the parent. For the second parent, the same process is repeated.

DR. VISHAL SHRIVASTAVA



Chromosome	Fitness Value
A	8.2
B	3.2
C	1.4
D	1.2
E	4.2
F	0.3

Fig 1.3: Roulette Wheel Selection

Implementation wise, we use the following steps –

- Calculate S = the sum of a fitnesses.
- Generate a random number between 0 and S.
- Starting from the top of the population, keep adding the finesses to the partial sum P, till P<S.
- The individual for which P exceeds S is the chosen individual.

Tournament Selection

In K-Way tournament selection, we select K individuals from the population at random and select the best out of these to become a parent. The same process is repeated for selecting the next parent. Tournament Selection is also extremely popular in literature as it can even work with negative fitness values.

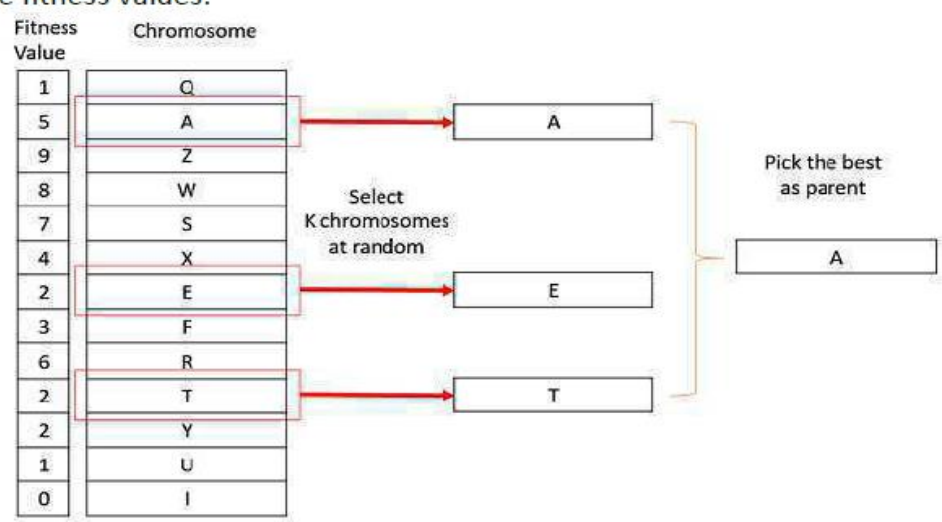


Fig 1.4: Tournament selection

Rank Selection

Rank Selection also works with negative fitness values and is mostly used when the individuals in the population have very close fitness values (this happens usually at the end of the run). This

leads to each individual having an almost equal share of the pie (like in case of fitness proportionate selection) as shown in the following image and hence each individual no matter how fit relative to each other has an approximately same probability of getting selected as a parent. This in turn leads to a loss in the selection pressure towards fitter individuals, making the GA to make poor parent selections in such situations.

In this, we remove the concept of a fitness value while selecting a parent. However, every individual in the population is ranked according to their fitness. The selection of the parents depends on the rank of each individual and not the fitness. The higher ranked individuals are preferred more than the lower ranked ones.

Chromosome	Fitness Value	Rank
A	8.1	1
B	8.0	4
C	8.05	2
D	7.95	6
E	8.02	3
F	7.99	5

Genetic Modeling: Inheritance Operator

In genetic algorithms, inheritance is the ability of modeled objects to mate, mutate (similar to biological mutation), and propagate their problem solving genes to the next generation, in order to produce an evolved solution to a particular problem. The selection of objects that will be inherited from in each successive generation is determined by a fitness function, which varies depending upon the problem being addressed.

The traits of these objects are passed on through chromosomes by a means similar to biological reproduction. These chromosomes are generally represented by a series of genes, which in turn are usually represented using binary numbers. This propagation of traits between generations is similar to the inheritance of traits between generations of biological organisms. This process can also be viewed as a form of reinforcement learning, because the evolution of the objects is driven by the passing of traits from successful objects which can be viewed as a reward for their success, thereby promoting beneficial traits.

Crossover Operator

Crossover is the process of taking two parent solutions and producing from them a child. After the selection (reproduction) process, the population is enriched with better individuals. Reproduction makes clones of good strings but does not create new ones. Crossover operator is applied to the mating pool with the hope that is creates a better offspring.

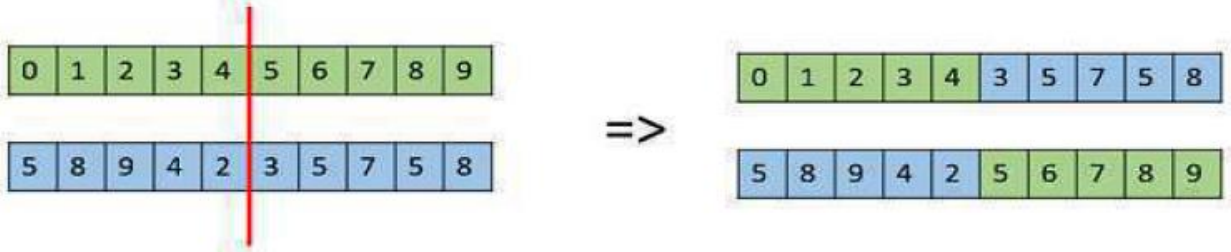
Crossover is a recombination operator that proceeds in three steps:

a. The reproduction operator selects at random a pair of two individual strings for the mating.
b. A cross site is selected at random along the string length.
c. Finally, the position values are swapped between the two strings following the cross site.

Some of cross over techniques are as follows:

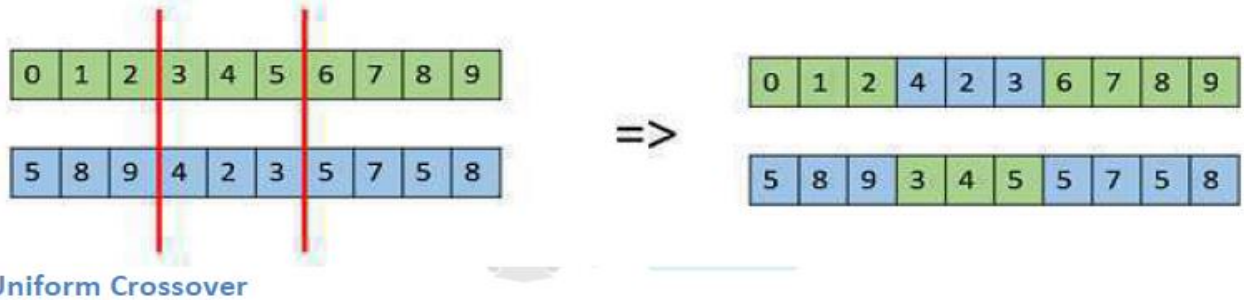
One Point Crossover

In this one-point crossover, a random crossover point is selected and the tails of its two parents are swapped to get new off-springs.



Multi Point Crossover

Multi point crossover is a generalization of the one-point crossover wherein alternating segments are swapped to get new off-springs.



Uniform Crossover

In a uniform crossover, we don't divide the chromosome into segments, rather we treat each gene separately. In this, we essentially flip a coin for each chromosome to decide whether or not it'll be included in the off-spring. We can also bias the coin to one parent, to have more genetic material in the child from that parent.

Inversion and Deletion Operator

In Inversion operator inverts the bits between two random sites.
Example: Chromosome 0100111 can be inverted to 0111001.
In Deletion two or three bits in random are selected and then deleted from chromosome.
Example: Chromosome 0100111 after deletion would be to 01001.

Mutation Operator

After a crossover is performed, mutation takes place. This is to prevent falling all solutions in population into a local optimum of solved problem. Mutation changes randomly the new

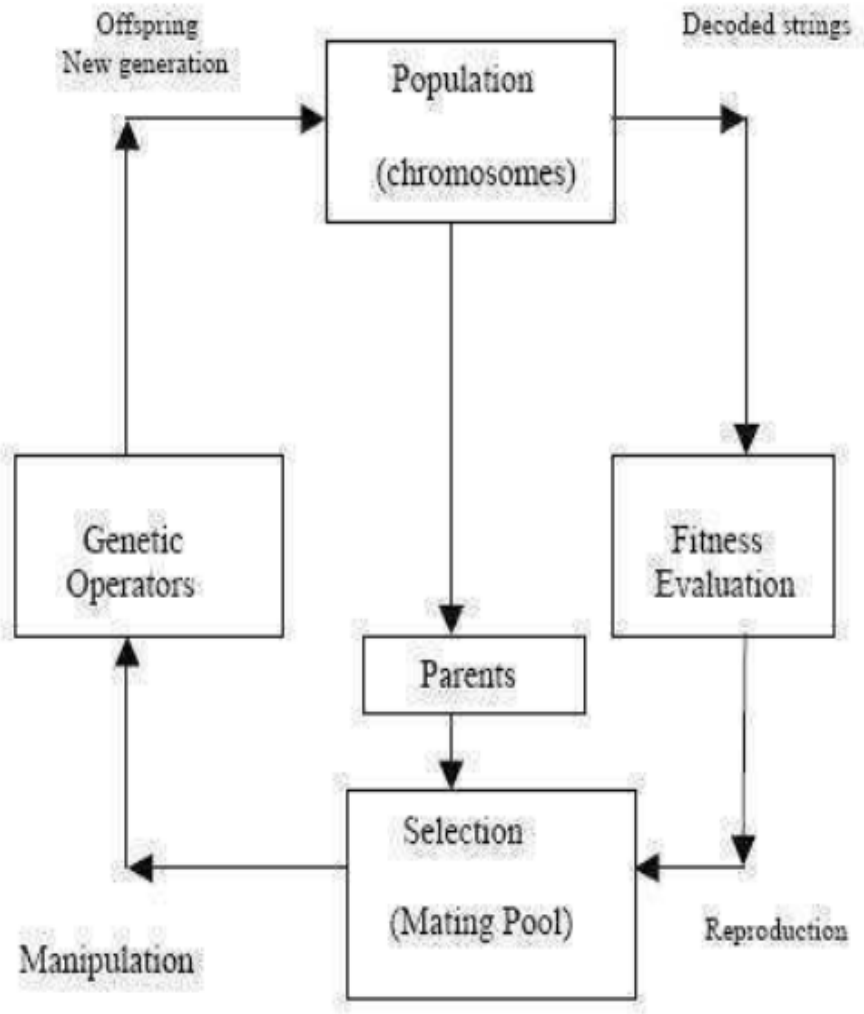


Fig 1.5 Generation Cycle

DR. VISHAL SH

Each cycle in Genetic Algorithms produces a new generation of possible solutions for a given problem. In the first phase, an initial population, describing representatives of the potential solution, is created to initiate the search process. The elements of the population are encoded into bit-strings, called chromosomes. The performance of the strings, often called fitness, is then evaluated with the help of some functions, representing the constraints of the problem. Depending on the fitness of the chromosomes, they are selected for a subsequent genetic manipulation process.

It should be notes that the selection process is mainly responsible for assuring survival of the best-fit individuals. After selection of the population strings is over, the genetic manipulation process consisting of two steps is carried out. In the first step, the crossover operation that recombines the bits (genes) of each two selected strings (chromosomes) is executed.

Convergence of GA

A genetic algorithm is usually said to converge when there is no significant improvement in the values of fitness of the population from one generation to the next. Examples of stopping criteria are generally, time limits placed on the GA run, generation limits, or if the algorithms finds a suitably low fitness individual, lower than a specified fitness threshold (in case we are minimizing fitness). There is no defined difference between stopping criteria and convergence criteria; the terms can be used interchangeably. Verifying that a GA has converged at global optima for a NP hard problem is impossible, unless you have a test data set for which the best

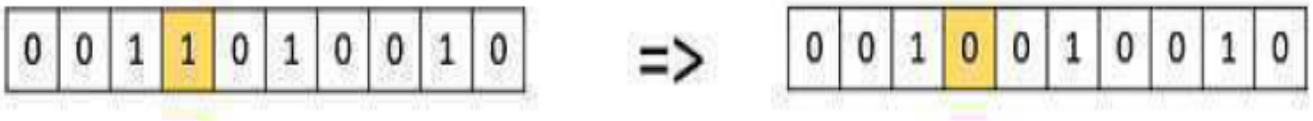
offspring. For binary encoding we can switch a few randomly chosen bits from 1 to 0 or from 0 to 1. Mutation can then be following:

Original offspring 1	1101111000011110
Original offspring 2	1101100100110110
Mutated offspring 1	1100111000011110
Mutated offspring 2	1101101100110110

The mutation depends on the encoding as well as the crossover. For example when we are encoding permutations, mutation could be exchanging two genes.

Bitwise operator

In this bit wise operator, we select one or more random bits and flip them. This is used for binary encoded GAs.



Generation Cycle:

A Genetic Algorithms operates through a simple cycle of stages:

i) Creation of a "; population" of strings,

ii) Evaluation of each string,

iii) Selection of best strings and

iv) Genetic manipulation to create new population of strings.

RGPVNOTES.IN

The cycle of a Genetic Algorithms is presented below:

solution is already known. The best you can do, is try out multiple runs of the GA, with different values of Mutation, Crossover probabilities, try out different fitness functions, crossover operators, and many variants of simple GA such as Elitism, Multi-Objective GA's etc.
The common GA terminating conditions are:

- When fixed number of generations are reached
- A optimal solution is obtained that satisfies the optimization criteria
- When successive GA iterations no longer produce better results
- Allocated budget (computational time / cost) reached

Application of GA

Genetic Algorithms are primarily used in optimization problems of various kinds, but they are frequently used in other application areas as well.
Following are some of the areas in which Genetic Algorithms are frequently used:

- **Optimization** – Genetic Algorithms are most commonly used in optimization problems wherein we have to maximize or minimize a given objective function value under a given set of constraints. The approach to solve Optimization problems has been highlighted throughout the tutorial.
- **Economics** – GAs are also used to characterize various economic models like the cobweb model, game theory equilibrium resolution, asset pricing, etc.
- **Neural Networks** – GAs are also used to train neural networks, particularly recurrent neural networks.
- **Parallelization** – GAs also have very good parallel capabilities, and prove to be very effective means in solving certain problems, and also provide a good area for research.
- **Image Processing** – GAs are used for various digital image processing (DIP) tasks as well like dense pixel matching.
- **Vehicle routing problems** – With multiple soft time windows, multiple depots and a heterogeneous fleet.
- **Scheduling applications** – GAs are used to solve various scheduling problems as well, particularly the time tabling problem.
- **Machine Learning** – as already discussed, genetics based machine learning (GBML) is a niche area in machine learning.
- **Robot Trajectory Generation** – GAs have been used to plan the path which a robot arm takes by moving from one point to another.
- **Parametric Design of Aircraft** – GAs have been used to design aircrafts by varying the parameters and evolving better solutions.
- **DNA Analysis** – GAs have been used to determine the structure of DNA using spectrometric data about the sample.
- **Multimodal Optimization** – GAs are obviously very good approaches for multimodal optimization in which we have to find multiple optimum solutions.
- **Traveling salesman problem and its applications** – GAs have been used to solve the TSP, which is a well-known combinatorial problem using novel crossover and packing strategies

Advances in GA: Implementation of GA using MATLAB

MATLAB has a wide variety of functions useful to the genetic algorithm practitioner.Given the versatility of MATLAB's high-level language, problems can be codedin m-files in a fraction of the time that it would take to create C or FORTRAN programs for the same purpose. Couple this with MATLAB's advanced data analysis,visualization tools and special purpose application domain toolboxes and the user ispresented with a uniform environment with which to explore the potential of geneticalgorithms.

The Genetic Algorithm Toolbox uses MATLAB matrix functions to build a setof versatile tools for implementing a wide range of genetic algorithm methods. TheGenetic Algorithm Toolbox is a collection of routines, written mostly in m-files,which implement the most important functions in genetic algorithms.

The main data structures used by MATLAB in the Genetic Algorithmtoolbox are:

- Chromosomes
- Objective function values
- Fitness values

Differences & similarities between GA & other traditional methods

GA is radically different from traditional optimization methods. Following are some of the differences of GA with other optimization techniques:

1. GA works with string coding of variables instead of variables. So that coding differentiate/categorize the search space even though the function is continuous.
2. GA works with population of points instead of single point.
3. In GA's previously found good information is emphasized using reproduction operator and propagated adaptively through crossover and mutation operators.
4. GA does not require any auxiliary information except the objective function values.
5. GA uses the probabilities in their operators. It narrows the search space as the search progresses is adaptive and is the unique characteristic of Genetic Algorithms.

Optimization

Optimization is a process that finds a best, or optimal, solution for a problem. The Optimization problems are centered around three factors :

• **An objective function :** which is to be minimized or maximized;

Examples:

In manufacturing, we want to maximize the profit or minimize the cost .

In designing an automobile panel, we want to maximize the strength.

• **A set of unknowns or variables :** that affect the objective function,

Examples:

In manufacturing, the variables are amount of resources used or the time spent.

In panel design problem, the variables are shape and dimensions of the panel.

• **A set of constraints:** that allows the unknowns to take on certain values but exclude others;

Examples:

1. In manufacturing, one constrain is, that all "time" variables to be non-negative.

• In the panel design, we want to limit the weight and put constrain on its shape.

An optimization problem is defined as : Finding values of the variables that minimize or maximize the objective function while satisfying the constraints.

Optimization Methods

Many optimization methods exist and categorized as shown below. The suitability of a method depends on one or more problem characteristics to be optimized to meet one or more objectives like :

− low cost,

− high performance,

− low loss

These characteristics are not necessarily obtainable, and requires knowledge about the problem.

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

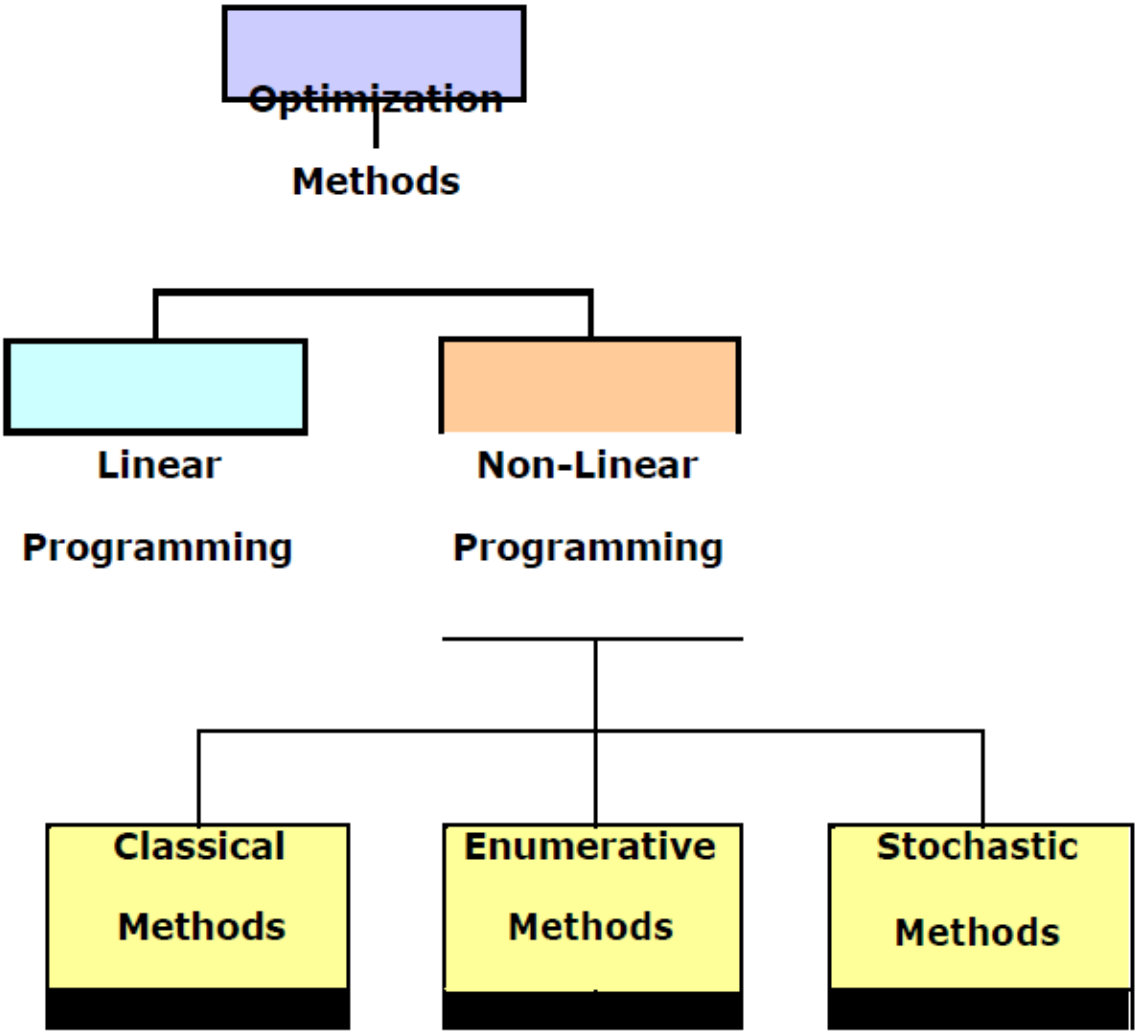


Fig. Optimization Methods

Linear Programming

Intends to obtain the optimal solution to problems that are perfectly represented by a set of linear equations; thus require a priori knowledge of the problem. Here the

− the functions to be minimized or maximized, is called *objective functions*,

− the set of linear equations are called *restrictions*.

− the *optimal solution*, is the one that minimizes (or maximizes) the objective function.

Example : "Traveling salesman", seeking a minimal traveling distance.

■ **Non- Linear Programming**

Intended for problems described by non-linear equations.

The methods are divided in three large groups:

Classical, Enumerative and Stochastic.

Classical search uses deterministic approach to find best solution. These methods requires knowledge of gradients or higher order derivatives. In many practical problems, some desired information are not available, means deterministic algorithms are inappropriate.

The techniques are subdivide into:

− Direct methods, e.g. Newton or Fibonacci −

Indirect methods.

Enumerative search goes through every point (one point at a time) related to the function's domain space. At each point, all possible solutions are generated and tested to find optimum solution. It is easy to implement but usually require significant computation. In the field of artificial intelligence, the enumerative methods are subdivided into two categories:

− Uninformed methods, e.g. Mini-Max algorithm

− Informed methods, e.g. Alpha-Beta and A* ,

Stochastic search deliberately introduces randomness into the search process. The injected randomness may provide the necessary impetus to move away from a local solution when searching for a global optimum. e.g., a gradient vector criterion for “smoothing” problems. Stochastic methods offer robustness quality to optimization process. Among the stochastic techniques, the most widely used are :

− Evolutionary Strategies (ES),

− Genetic Algorithms (GA), and

− Simulated Annealing (SA).

The ES and GA emulate nature’s evolutionary behavior, while SA is based on the physical process of annealing a material.

Search Optimization

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

Among the three Non-Linear search methodologies, just mentioned in the previous slide, our immediate concern is **Stochastic search** which means

− Evolutionary Strategies (ES),

−Genetic Algorithms (GA), and

−Simulated Annealing (SA).

The two other search methodologies, shown below, the *Classical* and the *Enumerative* methods, are first briefly explained. Later the *Stochastic methods* are discussed in detail. All these methods belong to Non-Linear search.

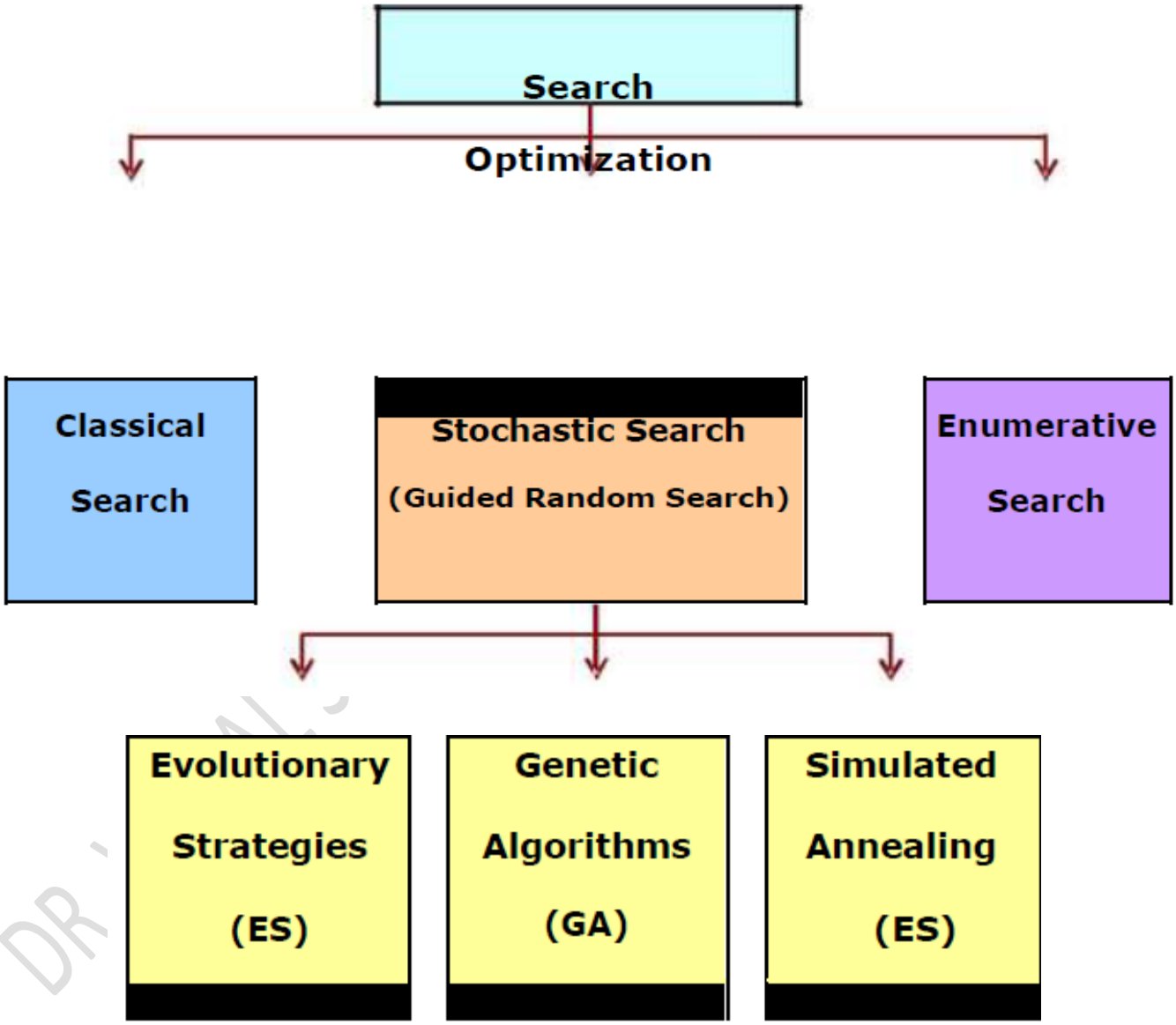


Fig Non- Linear search methods

Classical or Calculus based search

Uses deterministic approach to find best solutions of an optimization problem.

− the solutions satisfy a set of necessary and sufficient conditions of the optimization problem.

− the techniques are subdivide into **direct** and **indirect** methods.

Direct or Numerical methods :

− example: Newton or Fibonacci,

− tries to find extremes by "hopping" around the search space and assessing the gradient of the new point, which guides the search.

− applies the concept of "hill climbing", and finds the best local point by climbing the steepest permissible gradient.

− used only on a restricted set of "well behaved" functions.

• Indirect methods :

− does search for local extremes by solving usually non-linear set of equations resulting from setting the gradient of the objective function to zero.

− does search for possible solutions (function peaks), starts by restricting itself to points with zero slope in all directions

Enumerative Search

Here the search goes through every point (one point at a time) related to the function's domain space.

− At each point, all possible solutions are generated and tested to find optimum solution.

− It is easy to implement but usually require significant computation.

Thus these techniques are not suitable for applications with large domain spaces.

In the field of artificial intelligence, the enumerative methods are subdivided into two categories : Uninformed and Informed methods.

• Uninformed or blind methods :

− example: Mini-Max algorithm,

− search all points in the space in a predefined order,

− used in game playing.

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

• Informed methods :

− example: Alpha-Beta and A* ,

− does more sophisticated search

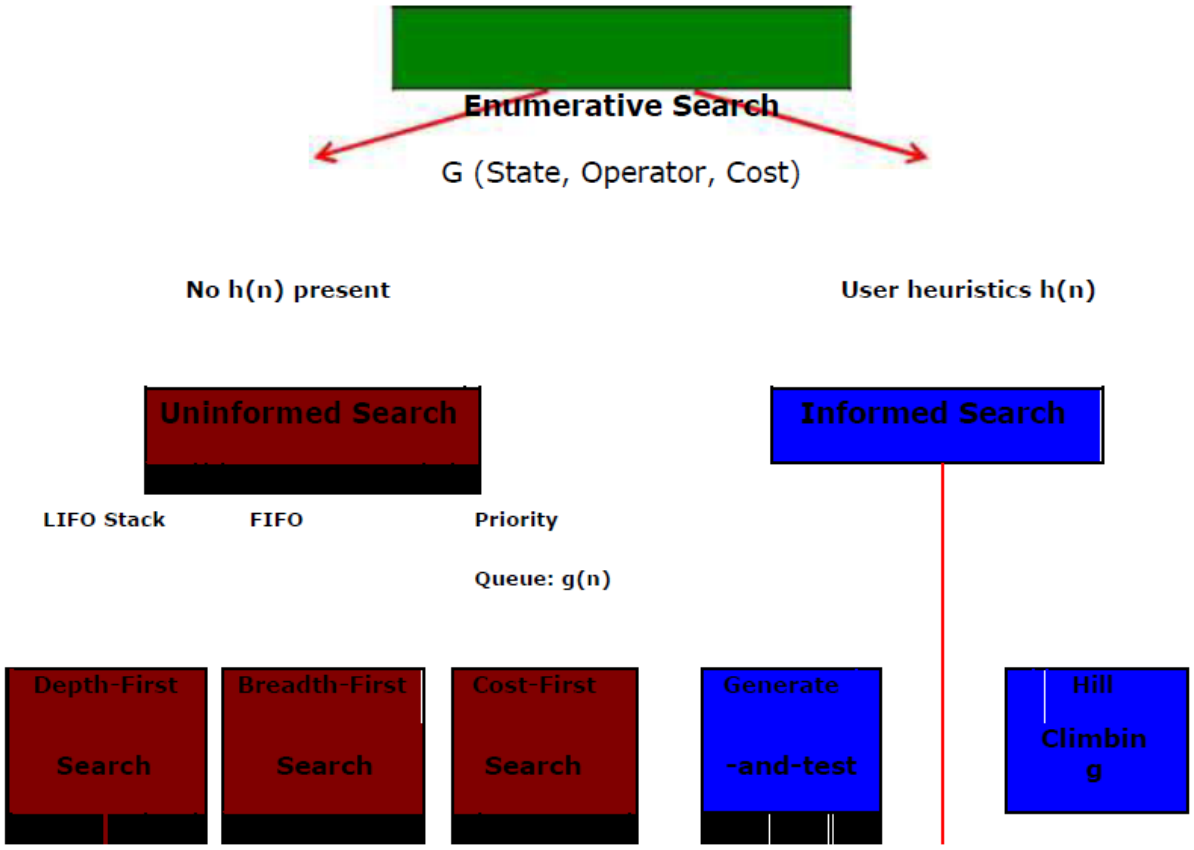
−uses domain specific knowledge in the form of a cost function or heuristic to reduce cost for search.

The Enumerative search techniques follows, the traditional search and control strategies, in the domain of Artificial Intelligence.

− The search methods explore the search space "intelligently"; means evaluating possibilities without investigating every single possibility.

− There are many control structures for search; the depth-first search and breadth-first search are two common search strategies.

− The taxonomy of search algorithms in AI domain is given below.



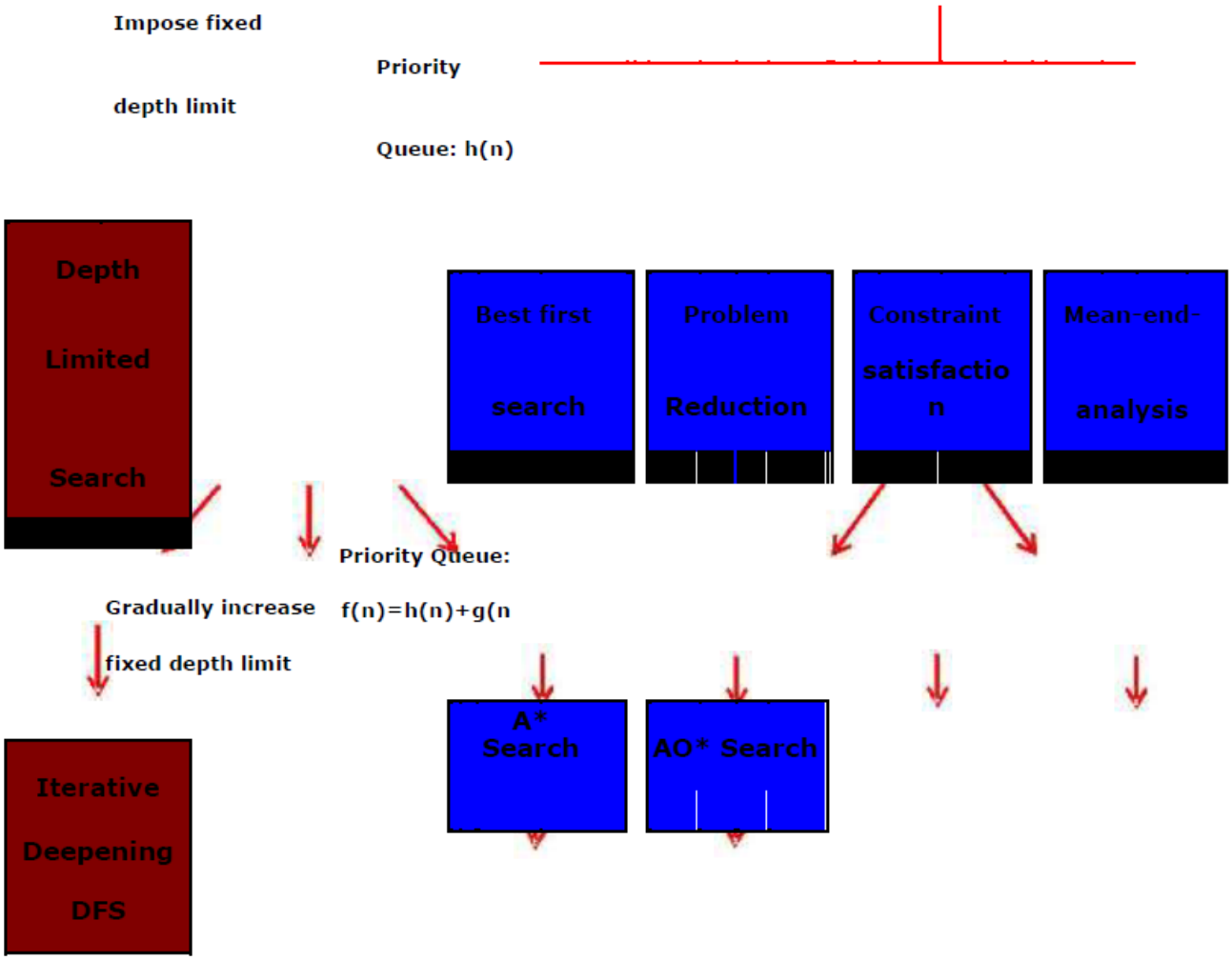


Fig. Enumerative Search Algorithms in AI Domain

Stochastic Search

Here the search methods, include heuristics and an element of randomness (non-determinism) in traversing the search space. Unlike the previous two search methodologies

− the stochastic search algorithm moves from one point to another in the search space in a non-deterministic manner, guided by heuristics.

− the stochastic search techniques are usually called Guided random search techniques.

The stochastic search techniques are grouped into two major subclasses:

− Simulated annealing and

− Evolutionary algorithms.

Both these classes follow the principles of evolutionary processes.

• Simulated annealing (SAs)

− uses a thermodynamic evolution process to search minimum energy states.

• Evolutionary algorithms (EAs)

− use natural selection principles.

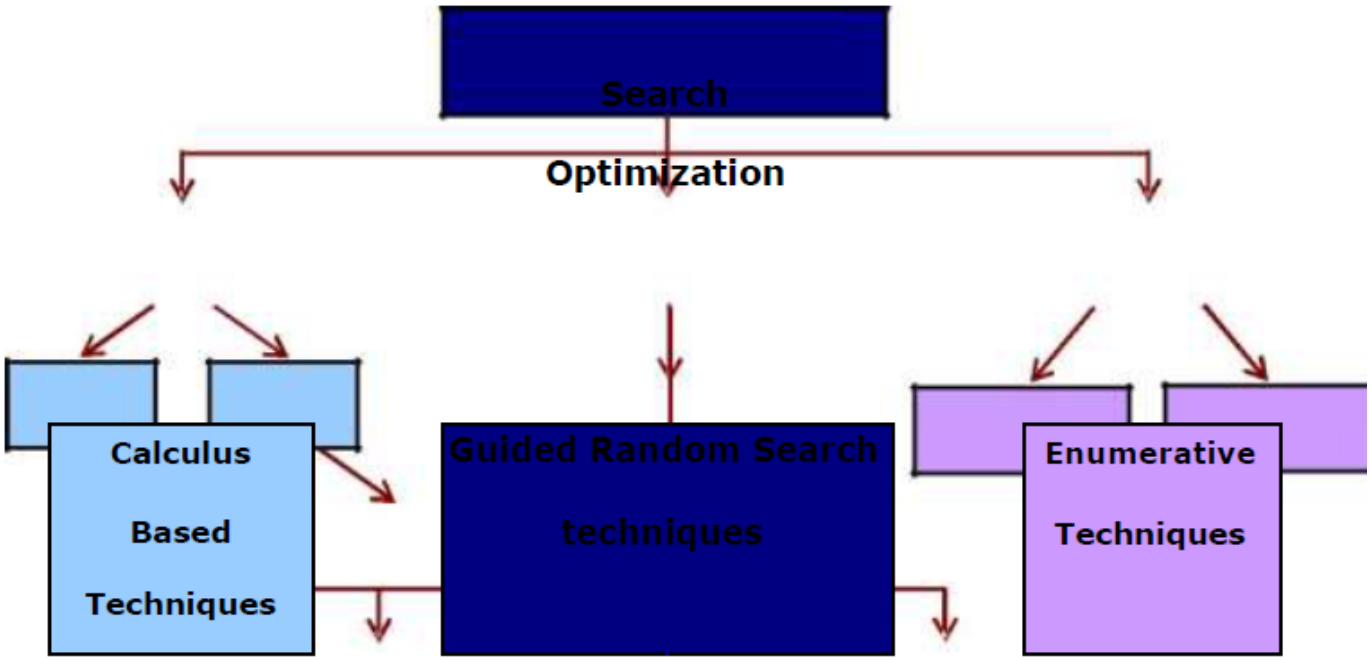
The search evolves throughout generations, improving the features of potential solutions by means of biological inspired operations.

− **Genetic Algorithms (GAs)** are a good example of this technique.

Taxonomy of Search Optimization

Fig. below shows different types of Search Optimization algorithms.

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT



Indirect method

Direct method

Uninformed Search

Informed Search

Newton

Finonacci

Tabu Search

Hill Climbing

Simulated Annealing

Evolutionary Algorithms

Genetic Programming

Genetic Algorithms

We are interested in Evolutionary search algorithms.

Our main concern is to understand the evolutionary algorithms :

- how to describe the process of search,
- how to implement and carry out search,
- what are the elements required to carry out search, and
- the different search strategies

The Evolutionary Algorithms include :

- Genetic Algorithms and
- Genetic Programming

1.3 Evolutionary Algorithm (EAs)

Evolutionary Algorithm (EA) is a subset of Evolutionary Computation (EC) which is a subfield of Artificial Intelligence (AI).

Evolutionary Computation (EC) is a general term for several computational techniques. Evolutionary Computation represents powerful search and optimization paradigm influenced by biological mechanisms of evolution : that of natural selection and genetic.

Evolutionary Algorithms (EAs) refers to Evolutionary Computational models using randomness and genetic inspired operations. EAs involve selection, recombination, random variation and competition of the individuals in a population of adequately represented potential solutions.

The candidate solutions are referred as chromosomes or individuals.

Genetic Algorithms (GAs) represent the main paradigm of Evolutionary Computation.

• GAs simulate natural evolution, mimicking processes the nature uses:

Selection, Crosses over, Mutation and Accepting.

• GAs simulate the survival of the fittest among individuals over consecutive generation for solving a problem.

Development History

EC = GP + ES + EP + GA

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

Evolutionary Computing	Genetic Programming	Evolution Strategies	Evolutionary Programming	Genetic Algorithms
Rechenberg	Koza	Rechenberg	Fogel	Holland
1960	1992	1965	1962	1970

1.4 Genetic Algorithms (GAs) - Basic Concepts

Genetic algorithms (GAs) are the main paradigm of evolutionary computing. GAs are inspired by Darwin's theory about evolution – the "survival of the fittest". In nature, competition among individuals for scanty resources results in the fittest individuals dominating over the weaker ones.

− GAs are the ways of solving problems by mimicking processes nature uses; ie., Selection, Crosses over, Mutation and Accepting, to evolve a solution to a problem.

− GAs are adaptive heuristic search based on the evolutionary ideas of natural selection and genetics.

− GAs are intelligent exploitation of random search used in optimization problems.

− GAs, although randomized, exploit historical information to direct the search into the region of better performance within the search space.

Biological Background – Basic Genetics

Every **organism** has a set of rules, describing how that organism is built. All living organisms consist of **cells**.

In each cell there is same set of **chromosomes.** Chromosomes are strings of DNA and serve as a model for the whole organism.

A chromosome consists of **genes**, blocks of DNA.

Each gene encodes a particular protein that represents a **trait** (feature), e.g., color of eyes.

Possible settings for a trait (e.g. blue, brown) are called **alleles**.

Each gene has its own position in the chromosome called its **locus**.

Complete set of genetic material (all chromosomes) is called a **genome**.

Particular set of genes in a genome is called **genotype**.

The physical expression of the genotype (the organism itself after birth) is called the **phenotype**, its physical and mental characteristics, such as eye color, intelligence etc.

When two organisms mate they share their genes; the resultant offspring may end up having half the genes from one parent and half from the other. This process is called **recombination** (cross over).

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

The new created offspring can then be mutated. **Mutation** means, that the elements of DNA are a bit changed. These changes are mainly caused by errors in copying genes from parents.

The **fitness** of an organism is measured by success of the organism in its life (survival).

Below shown, the general scheme of evolutionary process in genetic along with pseudo-code.

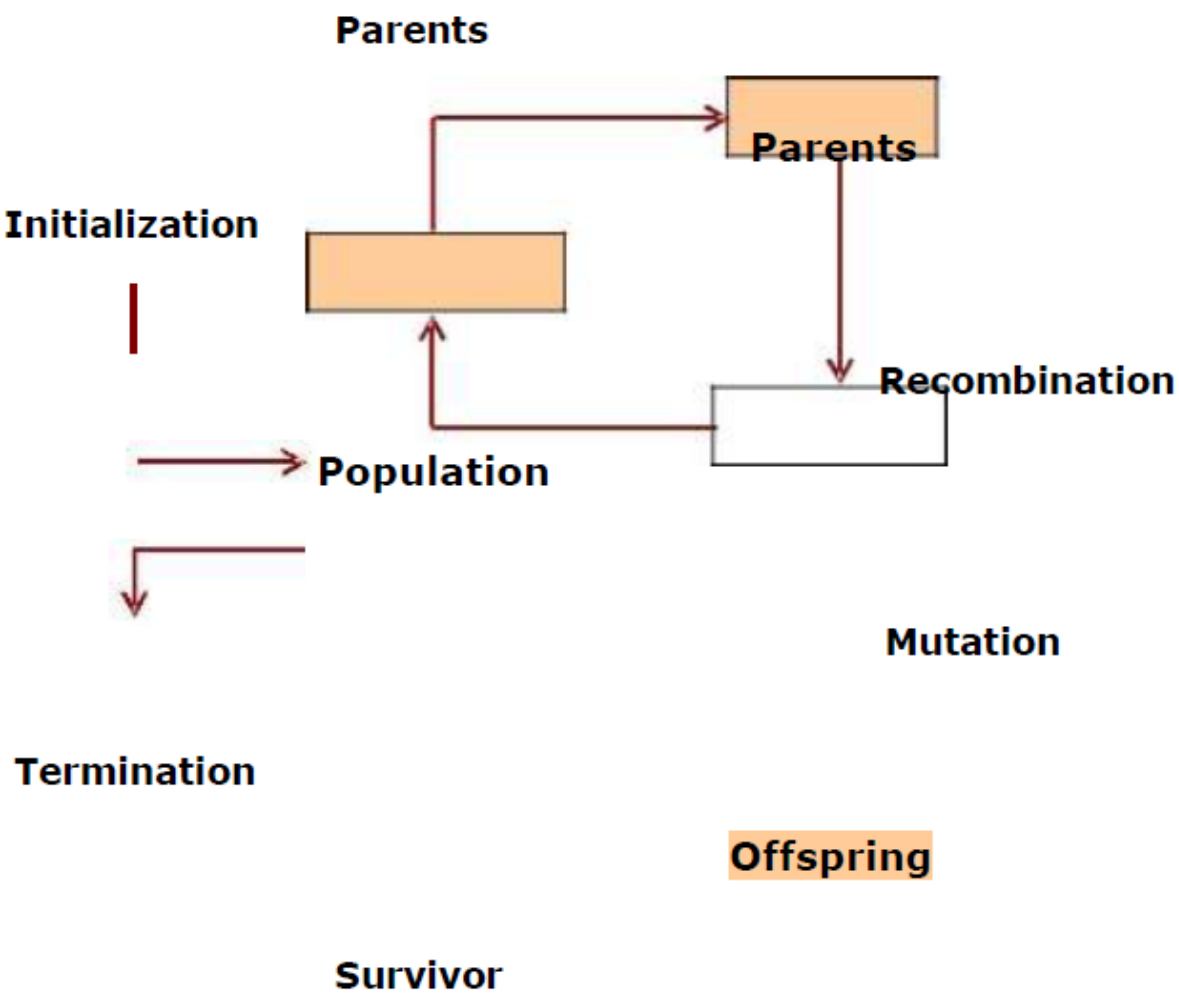


Fig. General Scheme of Evolutionary process

Pseudo-Code

```
BEGIN
INITIALISE population with random candidate solution.
EVALUATE each candidate;
```

REPEAT UNTIL (termination condition) is satisfied DO
■ SELECT parents;
■ RECOMBINE pairs of parents;
■ MUTATE the resulting offspring;
■ SELECT individuals or the next generation;
END.

Search Space

In solving problems, some solution will be the best among others. The space of all feasible solutions (among which the desired solution resides) is called **search space** (also called state space).

- Each point in the search space represents one **possible solution**.
- Each possible solution can be "marked" by its value (or **fitness**) for the problem.
- The GA looks for the **best solution** among a number of possible solutions represented by one point in the search space.
- Looking for a solution is then equal to looking for some extreme value (minimum or maximum) in the search space.
- At times the search space may be well defined, but usually only a few points in the search space are known.

In using GA, the process of finding solutions generates other points (possible solutions) as evolution proceeds.

- Working Principles

Before getting into GAs, it is necessary to explain few terms.

- Chromosome : a set of genes; a chromosome contains the solution in form of genes.
- Gene : a part of chromosome; a gene contains a part of solution. It determines the solution. e.g. 16743 is a chromosome and 1, 6, 7, 4 and 3 are its genes.
- Individual : same as chromosome.
- Population: number of individuals present with same length of chromosome.
- Fitness : the value assigned to an individual based on how far or close a individual is from the solution; greater the fitness value better the solution it contains.
- Fitness function : a function that assigns fitness value to the individual. It is problem specific.
- Breeding : taking two fit individuals and then intermingling there chromosome to create new two individuals.
- Mutation : changing a random gene in an individual.
- Selection : selecting individuals for creating the next generation.

Working principles :

DR. VISHAL SRIVASTAVA, PROFESSOR CSE, ACEIT

Genetic algorithm begins with a set of solutions (represented by chromosomes) called the population.
– Solutions from one population are taken and used to form a new population. This is motivated by the possibility that the new population will be better than the old one.
– Solutions are selected according to their fitness to form new solutions (offspring); more suitable they are, more chances they have to reproduce.
– This is repeated until some condition (e.g. number of populations or improvement of the best solution) is satisfied.

Outline of the Basic Genetic Algorithm

[Start] Generate random population of n chromosomes (i.e. suitable solutions for the problem).
[Fitness] Evaluate the fitness $f(x)$ of each chromosome x in the population.
[New population] Create a new population by repeating following steps until the new population is complete.
[Selection] Select two parent chromosomes from a population according to their fitness (better the fitness, bigger the chance to be selected)
[Crossover] With a crossover probability, cross over the parents to form new offspring (children). If no crossover was performed, offspring is the exact copy of parents.
[Mutation] With a mutation probability, mutate new offspring at each locus (position in chromosome).
(d) [Accepting] Place new offspring in the new population
• [Replace] Use new generated population for a further run of the algorithm
• [Test] If the end condition is satisfied, stop, and return the best solution in current population
• [Loop] Go to step 2
Note : The genetic algorithm's performance is largely influenced by two operators called crossover and mutation. These two operators are the most important parts of GA.

Flow chart for Genetic Programming

DR. VISHAL SRIVASTAVA, PROFESSOR CSE, ACEIT

Operators of Genetic Algorithm

Genetic operators used in genetic algorithms maintain genetic diversity. Genetic diversity or variation is a necessity for the process of evolution.

Genetic operators are analogous to those which occur in the natural world:
- **Reproduction** (or Selection) ;
- **Crossover** (or Recombination); and
- **Mutation**.

In addition to these operators, there are some parameters of GA.
One important parameter is **Population size**.
- Population size says how many chromosomes are in population (in one generation).
- If there are only few chromosomes, then GA would have a few possibilities to perform crossover and only a small part of search space is explored.
- If there are many chromosomes, then GA slows down.
- Research shows that after some limit, it is not useful to increase population size, because it does not help in solving the problem faster. The population size depends on the type of encoding and the problem.

3.1 Reproduction, or Selection

Reproduction is usually the first operator applied on population. From the population, the chromosomes are selected to be parents to crossover and produce offspring.

The problem is how to select these chromosomes ?

According to Darwin's evolution theory "survival of the fittest" – the best ones should survive and create new offspring.
- The Reproduction operators are also called Selection operators.
- Selection means extract a subset of genes from an existing population, according to any definition of quality. Every gene has a meaning, so one can derive from the gene a kind of quality measurement called **fitness function**. Following this quality (fitness value), selection can be performed.
- Fitness function quantifies the optimality of a solution (chromosome) so that a particular solution may be ranked against all the other solutions. The function depicts the closeness of a given 'solution' to the desired result.

Many reproduction operators exists and they all essentially do same thing. They pick from current population the strings of above average and insert their multiple copies in the mating pool in a probabilistic manner.
The most commonly used methods of selecting chromosomes for parents to crossover are :

- Roulette wheel selection,
- Boltzmann selection, -
Tournament selection,
- Rank selection
- Steady state selection.

Example of Selection

Evolutionary Algorithms is

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

the integer interval [0 , 31],
to maximize the function $f(x) = x^2$
i.e., $x = 0, 1, . . . 30, 31$.
with x
in
• The first step is encoding of chromosomes; use binary representation for integers; 5-bits are used to represent integers up to 31.
• Assume that the population size is 4.
• Generate initial population at random. They are chromosomes or genotypes; e.g., 01101, 11000, 01000, 10011.
• Calculate fitness value for each individual.
• Decode the individual into an integer (called phenotypes),
01101 → 13; 11000 → 24; 01000 → 8; 10011 → 19;
– Evaluate the fitness according to $f(x) = x^2$,
13 → 169; 24 → 576; 8 → 64; 19 → 361.
4. Select parents (two individuals) for crossover based on their fitness in p_i. Out of many methods for selecting the best chromosomes, if roulette-wheel selection is used, then the probability of the ith string in the population is $p_i = F_i / (\sum_{j=1}^{n} F_j)$, where
F_i is fitness for the string i in the population, expressed as $f(x)$
p_i is probability of the string i being selected,

4. is no of individuals in the population, is population size, $n=4$
$n * p_i$ is expected count

The string no 2 has maximum chance of selection

Roulette wheel selection (Fitness-Proportionate Selection)
Roulette-wheel selection, also known as Fitness Proportionate Selection, is a genetic operator, used for selecting potentially useful solutions for recombination.
In fitness-proportionate selection :
- the chance of an individual's being selected is proportional to its fitness, greater or less than its competitors' fitness.
- conceptually, this can be thought as a game of Roulette.

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

HOW ARE GENETIC ALGORITHMS DIFFERENT FROM TRADITIONAL METHODS?

In order for genetic algorithms to surpass their more traditional cousins in the quest for robustness, GAs must differ in some very fundamental ways. Genetic algorithms are different from more normal optimization and search procedures in four ways:

1. GAs work with a coding of the parameter set, not the parameters themselves.
2. GAs search from a population of points, not a single point.
3. GAs use payoff (objective function) information, not derivatives or other auxiliary knowledge.
4. GAs use probabilistic transition rules, not deterministic rules.

Genetic algorithms require the natural parameter set of the optimization problem to be coded as a finite-length string over some finite alphabet. As an example, consider the optimization problem posed in Fig. 1.5. We wish to maximize the function $f(x) = x^2$ on the integer interval [0, 31]. With more traditional methods we would be tempted to twiddle with the parameter x, turning it like the vertical hold knob on a television set, until we reached the highest objective function value. With GAs, the first step of our optimization process is to code the parameter x as a finite-length string. There are many ways to code the x parameter, and Chapter 3 examines some of these in detail. At the moment, let's consider an optimization problem where the coding comes a bit more naturally.

Consider the black box switching problem illustrated in Fig. 1.6. This problem concerns a black box device with a bank of five input switches. For every setting of the five switches, there is an output signal f, mathematically $f = f(s)$,

DR. VISHAL SHRIV

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

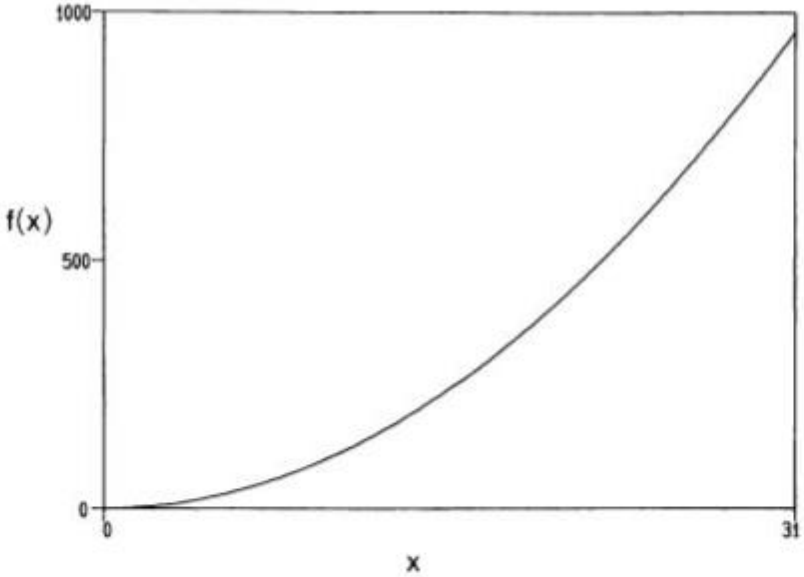


FIGURE 1.5 **A simple function optimization example, the function $f(x) = x^2$ on the integer interval [0, 31].**

where s is a particular setting of the five switches. The objective of the problem is to set the switches to obtain the maximum possible f value. With other methods of optimization we might work directly with the parameter set (the switch settings) and toggle switches from one setting to another using the transition rules of our particular method. With genetic algorithms, we first code the switches as a finite-length string. A simple code can be generated by considering a string of five 1's and 0's where each of the five switches is represented by a 1 if the switch is on and a 0 if the switch is off. With this coding, the string 11110 codes the setting where the first four switches are on and the fifth switch is off. Some of the codings introduced later will not be so obvious, but at this juncture we acknowledge that genetic algorithms use codings. Later it will be apparent

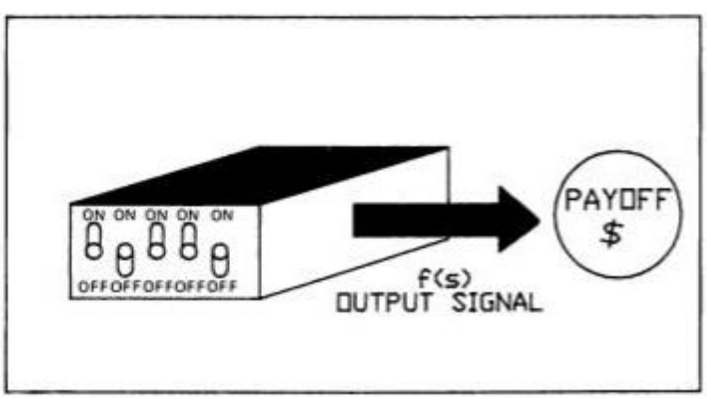


FIGURE 1.6 **A black box optimization problem with five on-off switches illustrates the idea of a coding and a payoff measure. Genetic algorithms only require these two things: they don't need to know the workings of the black box.**

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT

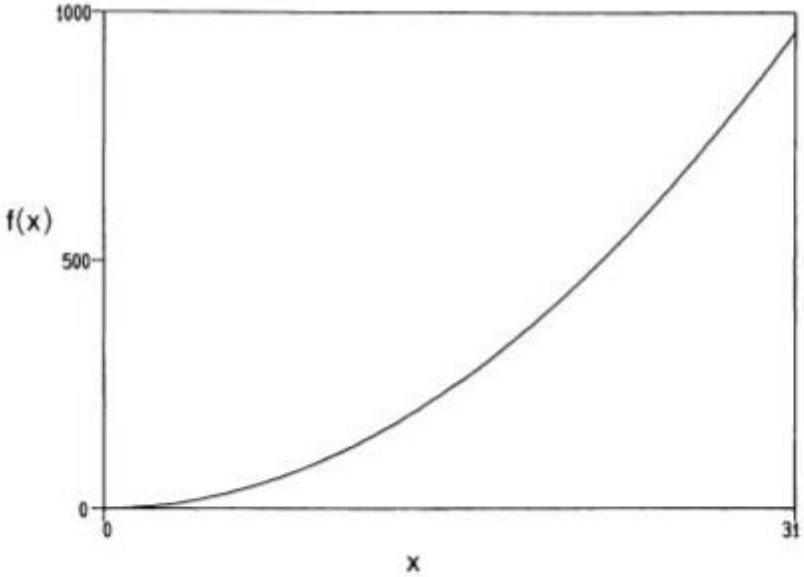


FIGURE 1.5 **A simple function optimization example, the function $f(x) = x^2$ on the integer interval [0, 31].**

where s is a particular setting of the five switches. The objective of the problem is to set the switches to obtain the maximum possible f value. With other methods of optimization we might work directly with the parameter set (the switch settings) and toggle switches from one setting to another using the transition rules of our particular method. With genetic algorithms, we first code the switches as a finite-length string. A simple code can be generated by considering a string of five 1's and 0's where each of the five switches is represented by a 1 if the switch is on and a 0 if the switch is off. With this coding, the string 11110 codes the setting where the first four switches are on and the fifth switch is off. Some of the codings introduced later will not be so obvious, but at this juncture we acknowledge that genetic algorithms use codings. Later it will be apparent

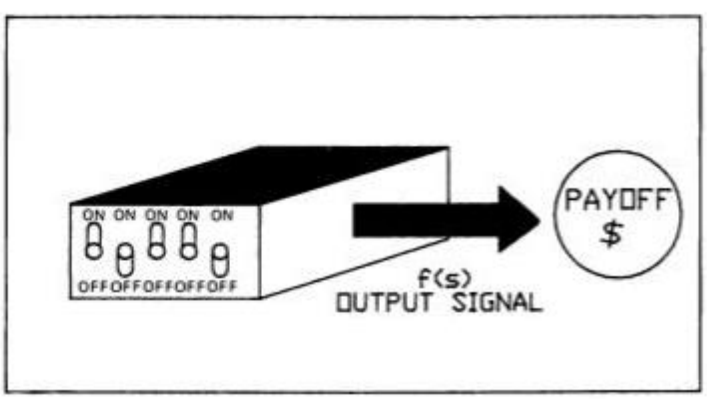


FIGURE 1.6 **A black box optimization problem with five on-off switches illustrates the idea of a coding and a payoff measure. Genetic algorithms only require these two things: they don't need to know the workings of the black box.**

that genetic algorithms exploit coding similarities in a very general way; as a result, they are largely unconstrained by the limitations of other methods (continuity, derivative existence, unimodality, and so on).

In many optimization methods, we move gingerly from a single point in the decision space to the next using some transition rule to determine the next point. This point-to-point method is dangerous because it is a perfect prescription for locating false peaks in multimodal (many-peaked) search spaces. By contrast, GAs work from a rich database of points simultaneously (a population of strings), climbing many peaks in parallel; thus, the probability of finding a false peak is reduced over methods that go point to point. As an example, let's consider our black box optimization problem (Fig. 1.6) again. Other techniques for solving this problem might start with one set of switch settings, apply some transition rules, and generate a new trial switch setting. A genetic algorithm starts with a population of strings and thereafter generates successive populations of strings. For example, in the five-switch problem, a random start using successive coin flips (head = 1, tail = 0) might generate the initial population of size $n = 4$ (small by genetic algorithm standards):

```
01101
11000
01000
10011
```

After this start, successive populations are generated using the genetic algorithm. By working from a population of well-adapted diversity instead of a single point, the genetic algorithm adheres to the old adage that there is safety in numbers; we will soon see how this parallel flavor contributes to a genetic algorithm's robustness.

Many search techniques require much auxiliary information in order to work properly. For example, gradient techniques need derivatives (calculated analytically or numerically) in order to be able to climb the current peak, and other local search procedures like the greedy techniques of combinatorial optimization (Lawler, 1976; Syslo, Deo, and Kowalik, 1983) require access to most if not all tabular parameters. By contrast, genetic algorithms have no need for all this auxiliary information: GAs are blind. To perform an effective search for better and better structures, they only require payoff values (objective function values) associated with individual strings. This characteristic makes a GA a more canonical method than many search schemes. After all, every search problem has a metric (or metrics) relevant to the search; however, different search problems have vastly different forms of auxiliary information. Only if we refuse to use this auxiliary information can we hope to develop the broadly based schemes we desire. On the other hand, the refusal to use specific knowledge when it does exist can place an upper bound on the performance of an algorithm when it goes head to head with methods designed for that problem. Chapter 5 examines ways to use nonpayoff information in so-called knowledge-directed genetic algorithms; however, at this juncture we stress the importance of the blindness assumption to pure genetic algorithm robustness.

Unlike many methods, GAs use probabilistic transition rules to guide their search. To persons familiar with deterministic methods this seems odd, but the use of probability does not suggest that the method is some simple random search; this is not decision making at the toss of a coin. Genetic algorithms use random choice as a tool to guide a search toward regions of the search space with likely improvement.

Taken together, these four differences—direct use of a coding, search from a population, blindness to auxiliary information, and randomized operators—contribute to a genetic algorithm's robustness and resulting advantage over other more commonly used techniques. The next section introduces a simple three-operator genetic algorithm.

DR. VISHAL SHRIVASTAVA, PROFESSOR CSE, ACEIT